Personal memory export

Dataset: `andrew-bydlon-memory` · **Exported:** 2026-10-28 15:45 EDT · **Entries:** 10

The ten entries below are the ones `list_memories` returned for this store, in the order the tool presents them (newest first). Bodies are reproduced verbatim — no truncation. All stored documents are single-chunk (`memory_truncated: false`), except where noted.

> **How the bodies were obtained.** `list_memories` only returns ~200-character previews, so each full body was recovered by locating the document through semantic search (`search_dataset` / `search_memory` on the same store) and matching on its `memory_ts`. Seven of ten resolved; three did not resolve under any query — including exact-phrase probes on their own preview text. Their previews are recorded in place of a body.

---

1 · project-summary · 2026-10-27 18:15:44 UTC

session `7d00305a465a600c` · **project** `mcp-fleet`

Full body not retrievable through the MCP surface. Only the preview is available: "*This session was initiated to execute task-2 o…*" (topics: refactoring, python-layout, mcp-servers, codebase-reorganization). Neither `search_dataset` nor `search_memory` returns this document — see Notes.

---

2 · project-summary · 2026-10-26 19:50:33 UTC

session `1d7f7f1250307fec` · **project** `multimodalrag`

Full body not retrievable through the MCP surface. Preview: "*Investigated why `GLM-5.pdf` showed no prog…*" (topics: pdf-embedding, contextual-retrieval, pipeline-debug, stalled-job). See Notes.

---

3 · gotcha · 2026-10-26 15:11:27 UTC

session `—` · **project** `glm-sglang-serving`

DSv4.1-Flash-1017 hicache boot, sglang build sha256:e3a4f060 (CUDA 13.0.3), scs04 4xH200 TP4/EP4, mf 0.8, Oct 26 2026. CORRECTION of session's earlier hypothesis: the ~27-min boot crash is HOST-SIDE (cgroup) exhaustion, NOT GPU. User's traceback: after "Allocating 70.84 GB host memory for V4 paged pool 'deepseek_v4_c2' (layers=3, pages=315342, item_bytes=74880)", headroom check "host 1241.4 GiB, cgroup 41.0 GiB" (node had 1.2TB free — the POD cgroup was the wall), then ValueError in DeepSeekV4PagedHostPool: "Not enough host memory for V4 paged pool deepseek_v4_c2_indexer. Requesting 8.23 GB but only have 8.12 GB free" — short by 0.11 GB. Ratio-3 takes the _dsv4_low_ratio_entries assembly path (pool composition differs from naive ratio×device model). Host need at ratio 3 ≈ 828-830 GiB vs 820Gi limit → 950Gi limit gives ~120 GiB margin → works. Ratio 4 estimated ≈ 1,030 GiB (scaled pools +33%, engram table fixed) → 950 NOT enough, needs ~1.05-1.1TB. Engram host table 96681 MiB is FIXED (invariant to hicache-ratio; verified identical at r4 and r3). Huge pages 96-100% on scs04 (THP madvise works). GPU side survived every stage of every attempt (avail mem=4.34 GB

after draft-verify capture) — mem-fraction 0.74/0.70 NOT required for this failure. User states mf 0.8 was their prior rung (supersedes older '0.70 mandatory' note). Operational quirks: KServe agent relaunches crashed sglang in-place → pod restarts stay 0, pod looks Running 3/4 while crash-looping, kserve log rotates each relaunch (grab root tracebacks within ~60s of the predicted crash mark or they're gone); teardown hangs 1-2 pids >60s post-SIGKILL but self-resolves, island drains to 4 MiB; "Load weight begin. avail mem=63.10/63.30 GB" reproduces byte-identical on DCGM-verified clean islands — benign budget figure, NOT dirty-GPU; post-"fired up" ~11-min silent warmup chain (FlashInfer autotune cached, DeepGEMM mostly cached, TRT-LLM fused-MoE autotune runs LIVE every boot — its cache is not PVC-mounted unlike TRITON_CACHE_DIR/SGLANG_DG_CACHE_DIR) precedes the uvicorn 8080 bind — "fired up" ≠ serving-ready, queue-proxy readiness is the real signal; tokenizer "still TokenizersBackend after retries" stall ~2.7 min each boot is benign; MLIS/KServe status-update conflicts freeze ISVC conditions at creation snapshot (use queue-proxy + logs); catalog model swaps (v8→v9→v10) kill the live pod mid-boot, knative route-churn warnings benign. Boot checkpoints (wall clock from pod start): host-table init ~9 min, capture end ~14 min, "fired up" ~15 min, warmup chain, draft-verify + pool-assembly crash window ~26.5-27 min.

---

4 · project-summary · 2026-10-26 14:05:12 UTC

session `54e8caa26b631380` · **project** `model-downloader`

Full body not retrievable through the MCP surface. Preview: "*The session addressed a colleague's report reg…*" (topics: deepseek, tr7, deployment, api-traffic, phantom-requests, oom). A sibling summary of the same session at **14:05:21** *is* retrievable — see Notes.

---

5 · project-summary · 2026-10-21 20:44:12 UTC

session `5f0e9c8c19527991` · **project** `multimodalrag`

```
[MEMORY INDEX | project: multimodalrag | type: session-summary | topics: search-
ranking,rrf,regression,context-enrichment,scoring]
```

What we did

Diagnosed a regression in multimodal search ranking where the query "mamba 2 architecture diagram" no longer returns the expected text+image pair at rank 1 (reranked) / rank 6 (unreranked). - Investigated why the specific document dropped in ranking; confirmed it was not due to the previous audit session breaking the index. - Identified a separate issue where the query "baby in the snow" returns a top result lacking context enrichment, despite the feature being enabled. - Recreated the index/search setup. Post-recreation, the "mamba" query returned at rank 7 (embedding-only, no reranking). - Verified and patched multiple scoring anomalies identified during the investigation. - Confirmed that the Rank-1 RRF (Reciprocal Rank Fusion) score calculation was incorrect, specifically regarding the interaction between `top_k` limits and rank contributions. - Established that the frontend search view should also utilize RRF scoring, not just the backend or API responses. - Investigated potential causes for residual scoring oddities, including the 1.5/1.0 scalars applied to the dataset and a potential 32 x 2 x 200 vector/dimension configuration.

Key decisions & rationale

1. **Frontend must use RRF**: The user explicitly corrected the assistant, stating, "Please ensure we fix that. It should use RFF on the front page too." This established that RRF is not just a backend aggregation step but a required part of the user-facing ranking logic.
2. **RRF Score Calculation Logic**: The user hypothesized that RRF scores are dependent on `top_k`. Specifically, if a document is rank 11 (index 10) in one list and rank 1 in another, its score under `top_k=10` would be 1/2 (from the rank 1 contribution only, assuming the other is cut off or handled differently), whereas under `top_k=20` it would include the 1/(1+10) contribution from rank 11. The user asked for verification and fixing of this behavior, granting the assistant final review authority ("You get the final review as DS").
3. **Context Enrichment Bug**: Acknowledged a bug where context is missing in top results for specific queries ("baby in the snow") even when context enrichment is enabled. This was flagged as a separate issue from the ranking regression.
4. **Dataset Scalars Hypothesis**: The user suggested that remaining scoring "weirdness" might be caused by the 1.5 / 1.0 scalars applied during dataset processing, rather than pure algorithmic errors.

Gotchas

- **Index Recreation Side Effects**: Recreating the index changed the rank of the target document from 6 (pre-recreation, with RRF) to 7 (post-recreation, embedding-only). This indicates that RRF was active in the pre-recreation state but not in the post-recreation test, or that the index state was not identical. The user clarified: "The number 7 is without reranking, only embedding." This distinction is critical for reproducing the bug; always specify whether RRF is active when reporting rank.
- **Frontend/Backend Scoring Mismatch**: The frontend was not applying RRF, leading to potentially different rankings than the backend. This caused confusion during debugging when comparing API results vs. UI display.
- **Context Enrichment Silent Failure**: Context enrichment can fail silently on specific results. The "baby in the snow" case showed a top result with no context despite the feature being enabled globally. This needs separate investigation from the ranking score issues.

How to proceed

- **Verify RRF `top_k` behavior**: Confirm that the RRF implementation correctly handles rank contributions when a document is outside the `top_k` window of one source but inside another. The user's hypothesis (score changes with `top_k`) needs to be validated against the actual implementation.
- **Fix frontend RRF application**: Ensure the frontend search view applies RRF scoring consistently with the backend.
- **Investigate context enrichment bug**: Determine why "baby in the snow" (and potentially other queries) miss context in top results. Check if this is related to specific metadata, embedding quality, or a bug in the enrichment pipeline.
- **Review dataset scalars**: Evaluate whether the 1.5/1.0 scalars applied to the dataset are causing unexpected score distributions or "weirdness" in rankings. Consider normalizing or adjusting these if they are skewing results.
- **Re-run cross-validation**: After patching the RRF and context issues, re-run the search queries for "mamba 2 architecture diagram" and "baby in the snow" to confirm expected rankings and context presence.

6 · project-summary · 2026-10-21 14:57:31 UTC

session `2e15df6c6f3cfea8` · **project** `pcai-llm-gateway`

```
[MEMORY INDEX | project: pcai-llm-gateway | type: session-summary | topics: usd-budget-presets,
helm-deployment, release-automation, distiller, sso-seeding]
```

What we did

- **USD Budget Support for Presets**: Extended the budget engine to support raw USD (`limit_usd`) budgets for `BudgetPreset` objects, previously restricted to token-only (`limit_tokens`).
  - **Core Logic** (`src/pcai_llm/core/budgets.py`): Added `split_preset_budgets(entries)` which validates budget entries, enforces XOR between token/USD limits, checks positivity and column-bound caps (`LIMIT_USD_MAX = 99_999_999.9999`, `LIMIT_TOKENS_MAX = 2**63 - 1`), and separates rows for `budgets` vs `cost_budgets` tables.
  - **Admin API** (`src/pcai_llm/api/routes_admin.py`): Updated `BudgetPreset` model to accept `limit_usd: float | None`. Profile create/update now uses the splitter for validation (400 on error). Key creation and profile attachment now seed both `budgets` and `cost_budgets` tables based on the split. `set_sso_default_profile` validates candidate profile presets at wire-time.
  - **Enrollment/SSO** (`src/pcai_llm/api/routes_enroll.py`): Updated `_read_profile_seed_tx` and `_seed_profile_defaults` to use the splitter. `_stamp_profile_grants` (used in SSO mint/rotate) now stamps both budget kinds. Added `ProfilePresetConfigError` exception. Invalid presets now result in a **503** response naming the specific profile, rather than an opaque 500.
- **Helm/Deployment Analysis**:
  - Confirmed `helm/templates/deployment.yaml:91` uses `image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"`.
  - Verified that pasting `image.tag: "v2.9.0"` into the values editor updates the image tag without requiring chart template changes.
  - Identified 10 new environment variables in the working tree `deployment.yaml` (e.g., `PCAI_LLM_DISTILLER_DISABLE_THINKING`, `PCAI_LLM_BUDGET_CACHED_EXEMPT_FRACTION`) vs. HEAD.
  - Noted that the currently deployed deployment is missing `PCAI_LLM_DISTILLER_DISABLE_THINKING` and `PCAI_LLM_DISTILLER_MAX_TOKENS`. However, `config.py:679` defaults `distiller_disable_thinking` to `True`, so the fix is active by default even if the env var is missing from the old chart.
- **Bug Fix & Release Prep**:
  - Fixed pre-existing test bug in `tests/test_canonical_retention.py`: `_reset_throttle` set `_last_canonical_prune = 0.0`, causing failures on hosts with uptime < 3600s. Changed sentinel to `-inf`. All 7 retention tests now pass.
  - Release gates green: `check_sql_schema.py`, `ruff`, and 1545 tests passed (56.84s).
  - Pods restarted via `kubectl rollout restart` on 2026-10-21T01:50Z using existing image digest `sha256:0bddfdcf...` (no rebuild).

Key decisions & rationale

1. **XOR Enforcement for Presets**: Enforced that a budget preset entry must specify *either* `limit_tokens` *or* `limit_usd`, not both or neither. This prevents ambiguous budget

configurations and aligns with the independent gate logic in the core budget engine.

2. **503 for Profile Config Errors**: Changed SSO mint/rotate failures due to invalid profile presets from 500 to 503. *Rationale*: A misconfigured profile is a server-side configuration issue that prevents service availability for that user/context, and returning 503 with the profile name aids debugging compared to an opaque 500.
3. **Values-Only Tag Override**: Decided to proceed with tag updates via Helm values editor rather than chart commits for the immediate deploy. *Rationale*: The template already supports `.Values.image.tag` override, and the critical distiller fix (`DISABLE_THINKING`) is enabled by code default, eliminating the need to ship the new env vars in this specific deploy step.
4. **Test Sentinel Fix**: Changed `_last_canonical_prune` initial value from `0.0` to `-inf` in tests. *Rationale*: `time.monotonic()` returns host uptime; on short-lived test containers or dev machines, `0.0` caused the throttle gate `now - _last < 3600` to incorrectly block the prune operation.

Gotchas

- **Helm Image Tag Override**: Do not assume chart changes are needed to change the image tag. `helm/templates/deployment.yaml` defaults to `.Chart.AppVersion` if `.Values.image.tag` is empty, but explicitly setting the tag in values overrides it.
- **Distiller Thinking Default**: `PCAI_LLM_DISTILLER_DISABLE_THINKING` is missing from the deployed environment, but the service behaves correctly because `config.py` defaults `distiller_disable_thinking` to `True`. Be aware that if this default changes, the old chart will not disable thinking.
- **Monotonic Clock in Tests**: When testing time-based throttles using `time.monotonic()`, initializing the "last run" timestamp to `0.0` will cause failures if the host uptime is less than the throttle interval. Use `-inf` to force the first run to execute.
- **Unpriced Models in USD Budgets**: Models with `NULL` cost in the ledger contribute `$0` to USD spend calculations. This means keys using unpriced models will not trigger USD budget gates, regardless of usage volume.

How to proceed

- **Distiller Failures**: Investigate ongoing distiller failures in production (4 failures, 8 "upstream produced no text" in logs). Backlog of 1–4 canonical rows is due.
- **Chart Sync**: Consider updating the Helm chart templates to include the 10 new environment variables identified in the working tree, especially if relying on env-var overrides rather than code defaults in the future.
- **v2.9.0 Release**: Execute `./automation.sh v2.9.0` to bump versions (`helm/Chart.yaml`, `pyproject.toml`, `appVersion`), build/push image, and package chart. Note that the working tree has 86 uncommitted files and no git remote configured.
- **Verification**: Confirm that the new USD budget preset functionality works end-to-end in the deployed v2.9.0 (or current) version, specifically testing profile-stamped keys and SSO self-mint paths.

---

7 · project-summary · 2026-10-21 03:33:57 UTC

session `e368f789c17f5ccb` · **project** `sqlhandler`

```
[MEMORY INDEX | project: sqlhandler | type: session-summary | topics: audit,rw-surfaces,semantic-catalog,admin-policy,mint-keys,identity-gate,helm-config,repr-bug]
```

What we did

Read-only audit of the SQLhandler repo at `/home/andrew/Code/HPE/SQLhandler` (Python MCP server + Starlette web UI + Helm chart) targeting three user-facing write flows. No files were edited. Subagent "audit-rw-surfaces" operated under zero-write-scope, reporting to team lead via `send_message`.

Files audited: `src/sqlhandler/webui.py` (1997 L), `server.py` (3948 L), `admin_keys.py` (344 L), `policy.py` (1153 L), `oidc_sso.py` (451 L), `saved.py`, `identity.py`, `engine.py`, `ui/index.html` (2475 L), `helm/templates/deployment.yaml` (451 L), `helm/values.yaml` (785 L), `helm/local/values.g2.yaml` (663 L), `helm/templates/configmap.yaml`.

Pytest: baseline 1390 passed / 12 skipped / 0 failed

---

8 · project-summary · 2026-10-19 16:05:10 UTC

session `ebdf9791eadad796` · **project** `deployments`

```
[MEMORY INDEX | project: deployments | type: session-summary | topics: dsv41, sglang, hicache, l3-storage, 8xh200, sprocket, deepseek-v4.1, file-backend]
```

What we did

Research session (read-only, no cluster changes) to determine whether the operator's hypothesis is correct: DeepSeek-V4.1-Flash does NOT suffer the GLM-5.3-Flash "mamba issue" that blocks L3 file-backed HiCache, and therefore the new 8xH200 "sprocket" stack can safely enable `--hicache-storage-backend file` on the 28 TB `/scratch` NVMe volume.

Deliverable written: `.scratch/2026-10-19-dsv41-sproket/research-l3-hicache.md` (19,217 chars, includes lead follow-up appendix).

Key evidence gathered from SGLang v0.5.22 source (fetched via `raw.githubusercontent.com/sgl-project/sglang/v0.5.22/...`):

- **V4.1 has NO mamba/KDA/linear-attention layers.** `configs/hybrid_arch.py:145-156` (`mambaish_config`) checks `mamba2_config`, `hybrid_gdn_config`, `hybrid_kda_config`, `hybrid_lightning_config`, and the linear-attn registry. `configs/linear_attn_model_registry.py` (2,997 lines) contains zero deepseek entries.
- **V4.1 has a purpose-built HiCache path.** `hybrid_cache/hybrid_pool_assembler.py:1757-1980` defines `_DeepSeekV4Strategy` and `build_deepseek_v4_hicache_stack` (lines ~860-1180) with sidecar pools for every V4 component (FULL, SWA, C128).
- **File backend IS permitted for DSV4.** `arg_groups/hicache_hook.py:179-182` (`resolve_storage_layout_compatibility`) early-returns for non-mooncake/npu backends, so no layout rewrite blocks it. `arg_groups/kv_cache_hook.py:562`: `assert cfg.hicache_storage_backend == "file", "... supports --hicache-storage-backend=file only."` — this is a *positive* gate confirming file is the supported backend for the unified-page-envelope path.
- **DSV4 is treated as MLA for storage ownership.** `managers/cache_controller.py:722-724`: `is_mla_model = isinstance(..., DeepSeekV4TokenToKVPool)`. Line 565-570: `self.backup_skip =`

`(self.storage_config.is_mla_model and self.storage_config.tp_rank != 0)` — only rank 0 writes storage, so the GLM multi-rank MAMBA write-refusal bug cannot reproduce.

- **GLM L3 root cause confirmed** (from `docker-compose.yml` commented L3 block + lead's independent verification): `lru_file_evictor.py:83` `_is_storage_owner` makes rank 0 sole owner; `hybrid_cache_controller.py:1615` `should_backup()` returns True for `PoolName.MAMBA` on every rank (KDA state is TP-sharded); ranks 1-7 get refused at `lru_file_evictor.py:178-182`; restore feeds rank-0 shard to all ranks → silent corruption.
- **Cookbook** (`.scratch/src/v41cb.mdx`, "Agentic Long-Context Deployment" section) explicitly offers HiCache and Mooncake as KV-offload choices for V4.1.
- **Engram host table env:** `environ.py:1605` `SGLANG_ENABLE_DSV41_ENGRAM_HOST_TABLE = EnvBool(False)`, line 1612 `SGLANG_DSV41_ENGRAM_HOST_TABLE_LAYOUT = EnvStr("shared")`.
- **File backend sizing envs:** `environ.py:774-782`: `SGLANG_HICACHE_FILE_BACKEND_STORAGE_DIR`, `_MAX_SIZE`, `_EVICTION_RATIO` (default 0.9), `_MIN_FREE_SPACE` (default "0"), `_ENABLE_METADATA_CACHE` (default False), `_METADATA_TTL` (default 5.0).
- **Live server state** (`.scratch/server_info.json`): `hicache_storage_backend: null`, `hicache_ratio: 2`, `enable_hierarchical_cache: false`, `page_size: 256`, `kv_cache_dtype: fp8_e4m3`.

Key decisions & rationale

1. **Verdict: CORRECT (with caveats).** The operator's hypothesis that V4.1 does not suffer the GLM mamba L3 problem is source-verified. The MAMBA pool simply does not exist in V4.1's `PoolName` enum, the linear-attn registry has no DSV4 entry, and the MLA storage-ownership path (`backup_skip` on ranks ≠ 0) prevents the multi-rank write-refusal corruption.
2. **File backend is the *only* supported storage backend for V4's unified-page-envelope path** (`kv_cache_hook.py:562` hard-asserts `== "file"`). This is not a limitation to work around; it is the designed path.
3. **V4's C4_STATE / C128_STATE pools are NOT independently TP-sharded** in the way MAMBA is. They travel through the unified page envelope host pool, which is owned by a single rank for storage I/O. The GLM corruption mechanism (per-rank MAMBA write attempts → refusal → rank-0-only restore) has no analogue.
4. **`--hicache-size` is not supported for V4; only `--hicache-ratio` works.** Verified in `arg_groups/fields/memory.py:106-110` (both fields exist) but the V4 path in `kv_cache_configurator.py` uses ratio-based sizing.
5. **Report language:** Initial report written in German (assistant's locale drift on turn 2); lead follow-up appendix appended in German as well. The report is at `.scratch/2026-10-19-dsv41-sproket/research-l3-hicache.md`.

Gotchas

- **`require('fs')` fails in PTC run_code** — the sandbox uses ESM. Fix: `const fs = await import('node:fs')`. Hit immediately on first tool call.
- **GitHub code search API requires auth (HTTP 401)** — worked around by fetching the full git tree (`/git/trees/v0.5.22?recursive=1`) and fetching individual raw files.
- **`hiradix_cache.py` does not exist in v0.5.22** (404). The file was renamed/restructured; the equivalent logic lives in `unified_radix_cache.py` (164,12

---

9 · project-summary · 2026-10-18 13:54:59 UTC

session `f5029e8c1bbf0499` · **project** `deployments`

```
[MEMORY INDEX | project: deployments | type: session-summary | topics: pcai-g2, williams-college-talk, hardware-specs, pptx-extraction, presentation-prep]
```

What we did

Prepared one-page summary documents for a Williams College presentation (mixed faculty, dean, IT org). Andrew wanted concise extraction pages covering: overall mission, shared infrastructure, dev system specs, small/medium/large specs — to keep the talk short and leave room to demo the actual platform.

Source deck: `/home/andrew/Code/HPE/scratch_workspace/WilliamsCollege/PCAI Hardware Specs 2.pptx` (36,431,099 bytes, 60 slides). Presenter on slide 1: Wyatt Newcomb (wyatt@hpe.com), HPE Private Cloud AI, "Part of the NVIDIA AI Computing by HPE Portfolio."

Extraction method: Python `zipfile` + `xml.etree.ElementTree` to parse `ppt/slides/slideN.xml` text elements (namespace `http://schemas.openxmlformats.org/drawingml/2006/main`). Script written to `.scratch/pptx-extract/` per scratch-dirs convention; scratch dir cleaned after use.

Output: A 7,719-byte markdown file (saved to workspace, path in the truncated tool output) containing six one-pagers: 1. Overall Mission — Private Cloud AI (turnkey AI-optimized private cloud, NVIDIA AI Computing by HPE portfolio, privacy/self-contained) 2. Shared Infrastructure 3. Developer System specs (Control Node ×1, AI Worker Node ×1, Integrated File+Object Storage, Ubuntu, HPE AI Essentials with NVAIE, Data Services Connector (DSC) VM, HPE VM Essentials) 4. Small specs 5. Medium specs 6. Large specs

Deck is G2-focused (X10000 hardware) with some G1 material. Confidential marking: "Authorized HPE Partner Use Only."

Key decisions & rationale

1. **Target the PPTX directly rather than a RAG dataset.** The `mm-rag__list_datasets` and `mm-rag__search_datasets` tools were not available in this session (returned "unknown tool"). Fallback: filesystem walk of the working directory found the `.pptx` under `WilliamsCollege/`.
2. **One-pager format over full slide dump.** Andrew explicitly said "keep this part fairly short overall so I can show them the actual platform." The output condenses 60 slides into ~6 tight sections.
3. **Scratch-dirs discipline for the extraction script.** Wrote the Python parser to `.scratch/pptx-extract/`, logged in `.scratch/MANIFEST.md`, deleted after completion.

Gotchas

- **`mm-rag` MCP tools unavailable.** Symptom: `Error: unknown tool "mm-rag__list_datasets"`. Cause: those tools weren't registered in this session's tool set. Fix: fall back to direct filesystem access and Python-based PPTX parsing.
- **Inline Python via `run_code` with `child_process` fails.** The first attempt to run Python inline produced a syntax error because of newline/shell-escaping in the string. Fix: write the script to a file first, then execute it.
- **PPTX is a 36 MB zip.** The file is large because of embedded images; text extraction is cheap but the file must not be moved or locked (a `.~lock` file was present, indicating it was open in LibreOffice/PowerPoint).

How to proceed

- **Review and edit the one-pager markdown** before the talk. Andrew may want to adjust emphasis (e.g., lead with privacy for a faculty/dean audience vs. specs for IT).
- **Demo prep:** The one-pagers are the "talk track"; the actual PCAI platform demo is the second half of the session. Confirm the demo environment is reachable and the target workloads are staged.
- **If additional slides are needed** (e.g., network topology, licensing, support SLA), re-run the extraction script against the same PPTX path and extend the markdown.
- **Lock file cleanup:** `WilliamsCollege/.~lock.PCAI Hardware Specs 2.pptx#` still exists — close the file in whatever editor left it before further edits.

---

10 · note · timestamp unknown

session `—` · **project** `glm-sglang-serving`

```
[MEMORY INDEX | project: glm-sglang-serving | type: theme | topics: h200, gpu, nvlink, dcgm,
xid, gpu-placement, glm, sglang]
```

SE-G2 GPU fleet: topology, telemetry, placement, health

Current state

- Fleet: 2 nodes pcai-se-scs04/scs05.hst.lab × 8 NVIDIA H200 NVL = 16 GPUs, driver 570.133.20; dcgm-exporter in hpecp-gpu-operator. DCGM: GPU_UTIL, FB_USED/FB_FREE (MiB, no FB_TOTAL — compute used+free), GPU_TEMP, POWER_USAGE, MEM_COPY_UTIL, XID_ERRORS, NVLINK_BANDWIDTH_TOTAL (KiB/s). Attribution labels: exported_namespace/exported_pod/exported_container (plain pod/namespace = exporter itself).
- **NVLink truth (operator-pinned)**: each node = TWO 4-GPU islands — GPUs 0-3 and 4-7 (NV6 within, SYS across; NV18 assumption was wrong). Overlay pins gpuNvlinkDomains="[[0,1,2,3],[4,5,6,7]]"; UI env PROM_UI_GPU_NVLINK_DOMAINS='[[0,1,2,3],[4,5,6,7]]'. Verified live: a TP KServe job moved ~3.1 MiB/s NVLink exactly on scs04 GPUs 4-7 while 0-3 idled.
- XID 31 latched on ALL 8 scs04 GPUs (confirmed 2026-10-16; first latched ~Sep 1 20:56–21:26 UTC, undiscovered until Sep 8) — scs05 clean. Pods (deepseek-v41-flash-offload, glm-53-flash) 0 restarts: GPU errors accumulate WITHOUT killing CUDA contexts — don't conflate "pods up" with "GPUs healthy". Open: thermal vs ECC → maintenance window if persistent; GLM MlisPredictorMemoryCritical (>91% cgroup) may be downstream.

What was decided / what exists

- **GPU placement policy (deliberate, Aug-2026)**: the four RAG-side predictors on scs04 (qwen3-vl-embedding-8b, qwen3-vl-reranker-8b, fish-s2-pro, cohere-transcribe) are INTENTIONALLY stacked on ONE shared GPU (NVIDIA_VISIBLE_DEVICES= in each InferenceService spec.predictor.containers[0].env) — politeness, NOT misconfiguration; don't recommend splitting. MIG NOT available. Cost: time-slicing serializes the four vLLM processes (Running:0/Waiting:0, multi-second tails; heavy ingest competes with search). Preserving improvements: pin a NON-default device UUID; NVIDIA MPS (CUDA_MPS_PIPE_DIRECTORY, CUDA_MPS_ACTIVE_THREAD_PERCENTAGE as software-MIG); shorter time-slice; RawDeployment to drop queue-proxy.

- **nvlink-topology DaemonSet** (chart-only): initContainer (python:3.12-slim, privileged, /run/nvidia/driver ro hostPath) probes the driver-tree nvidia-smi → topo -m to emptyDir; push container re-pushes every 20s; detection once per pod lifetime = node boot, by design. GOTCHAS it survived: (1) nvidia-smi COLORIZES even when piped — ANSI breaks ^GPU(\d+)$ matching and creates PHANTOM cross-island edges (two 4-GPU islands merged into "1 island of 8"); strip with re.sub(r"\x1b[[0-9;]*[A-Za-z]", "", line) + regression test on exact bytes; when a domain expert says the data is wrong, trust them and fetch the RAW artifact. (2) The HPE/PCAI pushgateway runs --metric.timetolive=30s — push-once groups vanish; any pushgateway metric needs a refresh loop UNDER the TTL; diagnose via push_time_seconds + scrape_samples_scraped. (3) The nvcr.io/nvidia/driver image has NO python3/nvidia-smi — the real one is at /run/nvidia/driver (host tree); needs privileged + LD_LIBRARY_PATH at a CLEAN dir with ONLY libnvidia-ml/libnvidia-cfg (the tree's lib dir is GLIBC_PRIVATE-locked to glibc 2.39 → "undefined symbol: __nptl_change_stack_perm").
- **H200 vs RTX Pro 6000 (spec-fair, Oct 7 final)**: NVFP4 only helps MoE weights (attention/KV stay FP8); decode is bandwidth-bound — H200's 2.7× bandwidth wins 1:1 with FP4 weights. Coding: H200x4 159.1 c1 / 493-532 c4 / 1,955-2,038 c32 / 2,886-3,142 c64; RTX (HelixML, TP2 capped at 4 running) 153.5 c1 / 388.2 c4. c1 parity; c4 RTX ~1.5x on paper (per-GPU 194 vs 123-133); c32+ H200 wins ~2.6x (RTX can't run >8 requests); H200 wins prefill 1.3x; RTX = ~1/3 cost, ≤8-user niche. "De-speculated" framing retracted — both sides run EAGLE 5/1/6.
- **Sprocket 8×H200 posture (2026-09-15)**: TP8/EP8, mem_fraction_static=0.78, fp8 37.93 GB/rank + 1.23 MTP draft; dsa/tilelang, page_size=64, ctx 1,048,576; EAGLE adaptive 3-5; max_running_requests=64 (ready avail 13.51 GB vs 18.92 at 48 — tier states scale with max_running; 128 ⇒ ~14.5 GB tier states, confirming the 128-OOM math). L1 ~559 GB VRAM cache (KV 45.89 + DSA 4.17 + mamba 19.84 GB/rank), max_total_num_tokens=3,874,624; L2 HICACHE_GB=225/rank, split proportionally across the three hybrid pools. FABRIC→POSIX fd fallback lines = benign; fallback HICACHE_GB=200/rank if avail < ~100 GiB.
- **MLIS "Pause" = DELETE (2026-09-02 ~16:47 UTC)**: a session as nandita-goyal paused her diffusiongemma-26b-a4b-it-bf16 (still paused) AND Andrew's qwen-38-27b, glm-53-flash, cohere-transcribe-03-2026 → DeleteInferenceService → full knative teardown; her pending qwen35-122b-a10b-ng went Ready 83s after the last pause freed GPUs. All six isvcs recreated, Ready by 17:11. OPEN: cross-project pause authorization (platform team); pause = destructive delete + cold start. Audit: aioli-master logs (audit.go createAuditEvent; reconcile.go:320).

Notes

1. Three of the ten could not be expanded. The nearest retrievable siblings of the unresolved entries:

- *For entry 1 (mcp-fleet, task-2):* `2026-10-27T18:10:51` — "Initiated a 6-repo Python layout refactor in `/home/andrew/Code/HPE/mcp_servers` per `.scratch…`" (4,642 chars) and `18:11:48` — "src-layout-refactor, searxng-mcp, workbench-mcp, packaging-tests, hardlink-mesh, lint-drift" (2,396 chars).
- *For entry 2 (GLM-5.pdf):* `2026-10-26T17:00:55` — "User reported that a previously uploaded file **GLM-5.pdf** appears stuck at the 'embedding…' stage with no visible progress" (1,196 chars).

- *For entry 4 (model-downloader / tr7):* `2026-10-26T14:05:21` — "sglang, claude-code, anthropic-api, tr7, unexplained-traffic, seed-catalog" (5,305 chars).

2. This is the tool's list, not a strict timestamp sort. `list_memories` labels its output "newest first", but it sorts a Qdrant *scroll sample* — with 651 documents in the store it is not guaranteed to be the true newest-N. Concretely: the `2026-10-27T18:10` and `18:11` mcp-fleet summaries are timestamp-newer than entry 1 and were skipped, and a batch of `dsh-curation` summaries at `13:52–13:54` on 2026-10-27 is likewise absent.

3. A guaranteed-ordered dump is possible. The dataset is `has_password: false` and reachable read-only via the gateway. An exact export needs a `kubectl port-forward` to `rag-mcp-server` (the ingress SSO-gates `/api/datasets/.../export` from this host, and no cluster context is configured locally).